

LAB 3

Component Modelling & Architectural Pattern Selection

Self-Service Coffee Kiosk System

Name	MUKESH SHEREGAR
SRN	PES1UG25CS828
Section	I

1. Objective

Evaluate different architectural styles, select the most appropriate style for the assigned scenario, and create a UML component model showing modules, interfaces, and dependencies.

2. Scenario Review

The self-service coffee kiosk is designed for a busy café. Customers select a coffee type, choose a drink size, pay by credit card, and receive a printed receipt.

Functional Requirements

- Coffee types: Espresso, Americano, and Latte.
- Drink sizes: Small and Large.
- Credit-card payment only.
- Receipt printing with order details.

Technical Constraints

- Touch-screen interaction.
- Connection to receipt-printer hardware.
- Storage of menu data and pricing information.

3. Architectural Styles Compared

Style	Structure	Pros	Cons
Layered	Presentation, Business, Data	Separation of concerns; simple; maintainable.	Layer coupling; possible overhead.
Microservices	Independent deployable services	Independent scaling; fault isolation.	Operational complexity; network latency; consistency challenges.
Client-Server	Central server with multiple clients	Central control; simple deployment.	Single point of failure; scalability bottleneck.

4. Architecture Selection & Component Identification

Selected Architecture

We chose Layered Architecture for the Self-Service Coffee Kiosk System.

Reasons for Selection

1. Clear separation of concerns: Presentation, Business Logic, and Data/External Services can be kept separate.
2. Suitable for the scenario: the kiosk has a small, well-defined set of functions and does not require microservice-level operational complexity.
3. Controlled component interaction: payment, menu data, and printer access can be reached through defined interfaces.

Why the Other Styles Were Not Selected

Microservices

Microservices offer independent deployment and scaling, but the single kiosk has limited functionality. The additional service management, network communication, and data-consistency concerns are unnecessary for this scenario.

Client-Server

Client-server provides centralized control, but the kiosk requires direct interaction with a touch screen and receipt-printer hardware. Layering gives clearer separation of user interaction, business processing, and external services.

5. Components

Component	Layer	Responsibility
Touch Screen UI	Presentation	Displays menu, collects selections and shows payment status.
Order Manager	Business Logic	Creates/validates orders, calculates price and coordinates services.
Payment Service	Business Logic	Processes credit-card payments and returns transaction status.
Menu Database	Data	Stores coffee types, sizes and pricing.
Receipt Printer	External Service	Formats and prints the completed receipt.

6. Component Responsibilities

- Touch Screen UI receives customer input and presents system feedback.
- Order Manager acts as the central business-processing component.
- Payment Service isolates payment processing from the user interface.
- Menu Database provides consistent menu and pricing information.
- Receipt Printer handles physical receipt output.

7. Interfaces & UML Component Model

Required Interfaces

Interface	Provider	Consumer	Purpose
IOrderService	Order Manager	Touch Screen UI	Create order and return order information.
IPaymentService	Payment Service	Order Manager	Process credit-card payment.
IMenuService	Menu Database	Order Manager	Retrieve menu, sizes and prices.
IPrintService	Receipt Printer	Order Manager	Print completed receipt.

Text-Based UML Component Diagram

The following layout represents the component diagram using the required component and interface concepts. The provided interface is represented by a ball (○) and the required interface by a socket (⌋).

PRESENTATION LAYER

<<component>> Touch Screen UI

BUSINESS LOGIC LAYER

<<component>> Order Manager	<<component>> Payment Service
---	---

DATA & EXTERNAL SERVICES LAYER

<<component>> Menu Database	<<component>> Receipt Printer
---	---

Touch Screen UI	→ IOrderService →	Order Manager
Order Manager	→ IPaymentService →	Payment Service
Order Manager	→ IMenuService →	Menu Database
Order Manager	→ IPrintService →	Receipt Printer

Interaction Flow

- Customer selects coffee type and size on the Touch Screen UI.
- Order Manager receives the order and requests current pricing from the Menu Database.
- Order Manager sends the payment request to the Payment Service.
- After successful payment, Order Manager sends the order details to the Receipt Printer.
- Receipt Printer produces the final receipt.

UML Notation

- Components use the <<component>> stereotype.
- Provided services use ball notation; required services use socket notation.
- Dependencies may be shown using dashed arrows with <<use>>.
- Assembly connectors connect a required interface to a provided interface.

8. Written Architectural Justification

Architecture Selection: We chose Layered Architecture for the Self-Service Coffee Kiosk System.

Architectural Choice

The architecture separates the system into a Presentation Layer, Business Logic Layer, and Data/External Services Layer. This organization matches the kiosk's responsibilities and makes the components easier to understand and maintain.

Two Specific Reasons

9. Clear separation of concerns: the Touch Screen UI handles customer interaction, the Order Manager handles order processing, and the Menu Database and Receipt Printer handle data and external services.
10. Appropriate complexity: the kiosk has limited, well-defined functionality, so a layered design is sufficient without the operational overhead associated with multiple independently deployed microservices.

Security Advantage

The architecture limits direct access to critical services. Payment processing and menu data can be accessed through controlled business-logic interfaces, allowing validation before requests reach sensitive components.

Performance Benefit

The separation allows each layer to be optimized for its task. Touch interaction can remain responsive while order processing, menu queries, payment processing, and receipt output are handled by their respective components.

9. Requirement-to-Component Mapping

Requirement	Responsible Component
Touch-screen interaction	Touch Screen UI
Coffee type and size selection	Touch Screen UI + Menu Database
Order validation and total calculation	Order Manager
Credit-card payment	Payment Service
Receipt generation and printing	Receipt Printer
Menu and pricing storage	Menu Database

10. Final Review

- At least five components are present.
- Four interfaces are identified: IOrderService, IPaymentService, IMenuService, and IPrintService.
- Component interactions and data flow are documented.
- The selected architecture is justified using scenario-specific reasons.
- Security and performance implications are addressed.
- The final diagram and justification can be exported to PDF and submitted under Lab3 in the GitHub repository.

Conclusion

Layered Architecture is a suitable choice for the self-service coffee kiosk because it provides clear component boundaries, controlled interactions, and a straightforward structure for the required ordering, payment, menu, and receipt functions.